# Distributed Multi-Model Analytics for E-Commerce Data

Technical Report

**Ahourdet Donambi Thierry**
AUCA Big Data Analytics
View Source Code on GitHub

June 2026

**Abstract**

This project implements a complete analytics pipeline for a synthetic e-commerce dataset comprising 10k users, 5k products, 2M sessions, and 500k transactions over 90 days. The system leverages **MongoDB** for document-oriented entity storage (users, products, transactions), **HBase** as a wide-column store for high-velocity clickstream logs, and **Apache Spark** for distributed batch processing and cross-system integration. The architecture demonstrates that MongoDB excels at complex, ACID-compliant document operations, HBase is unmatched for sparse time-series event storage, and Spark acts as the unifying engine for large-scale, cross-store analytics. A secondary contribution is methodological: each analytical output is validated against the data generator, which establishes that several apparent findings are artefacts of uniform random assignment rather than genuine patterns. Distinguishing a correct mechanism from a real signal is treated here as a result in its own right.

## Executive Summary

Key outcomes of this distributed implementation include:

- A denormalised MongoDB schema with embedded category hierarchies and pre-computed lifetime summaries that enable sub-second user segmentation and revenue aggregations.
- An HBase schema with reversed-timestamp row keys and sparse column families that reduces "recent sessions" scans to $\mathcal{O}(1)$ prefix lookups.
- A PySpark co-occurrence matrix that identifies frequently co-viewed product pairs, and a Spark SQL query that aggregates revenue by region across nested JSON structures.
- An integrated funnel analysis combining HBase's intent signals (carts) with MongoDB's purchase records, showing that 64.9% of sessions reach a cart while completed purchases amount to 24.4% of that figure.

- A significance audit of every headline result against the data generator. Regional revenue is shown to rank user count rather than demand; product affinity is shown to depart from a random baseline yet still fail as a recommendation signal, because raw co-occurrence measures popularity rather than relatedness. The baseline is computed inside the pipeline and emitted alongside the result.

# 1 System Architecture Overview

The system is deployed using Docker Compose, with three core components orchestrated to separate storage from compute.

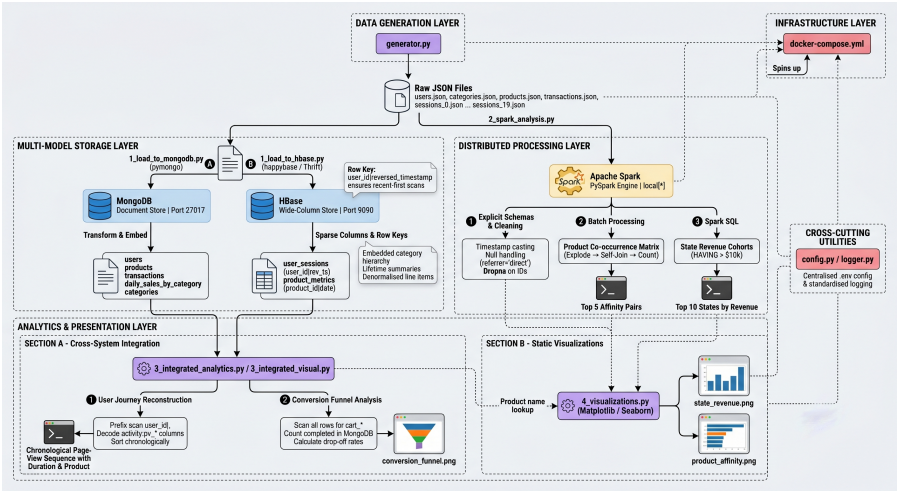


Figure 1: End-to-end data pipeline: data generation → ingestion into MongoDB/HBase → Spark processing → visualisation.

- **Data Generation** (`generator.py`) produces JSON files for users, categories, products, sessions (split into 20 chunks) and transactions.
- **Data Ingestion**:
  - MongoDB: `1_load_to_mongodb.py` transforms and loads entities with embedded summaries and materialised views.
  - HBase: `1_load_to_hbase.py` creates two tables (`user_sessions`, `product_metrics`) using Thrift and loads session data with sparse activity columns.
- **Processing**:
  - Apache Spark (`2_spark_analysis.py`) reads raw JSON, cleans data, computes product affinities against a random baseline, executes SQL analytics, and persists both to `results/` as CSV.
  - Integration scripts (`3_integrated_analytics.py`, `3_integrated_visual.py`) combine MongoDB and HBase data to produce funnel metrics and user-journey reconstructions.
- **Visualisation**: `4_visualizations.py` generates static charts with Matplotlib and Seaborn, reading the Spark stage from `results/` and the document stage from MongoDB. No chart values are hard-coded, so the figures regenerate with the pipeline.

# 2 Data Modeling & Storage (Part 1)

## 2.1 MongoDB – Document Model

### 2.1.1 Product Catalog (Embedded Category Hierarchy)

```
{
  "_id": "prod_00123",
  "name": "Wireless Gaming Mouse",
  "base_price": 129.99,
  "current_stock": 147,
  "is_active": true,
  "creation_date": ISODate("..."),
  "price_history": [ { "price": 149.99, "date": ISODate("...") }, ... ],
  "category": { "id": "cat_007", "name": "Electronics",
    "subcategory": { "id": "sub_007_01", "name": "Gaming Peripherals", "profit_margin":
    0.30 } },
  "performance_metrics": { ... }   // reserved: roll-up from HBase product_metrics
}
```

Collection: products

**Justification**: Products are read millions of times per day; embedding the full category path avoids a join and ensures fast catalog browsing. The `price_history` array preserves temporal price changes for trend analysis.

### 2.1.2 User Profiles (Embedded Lifetime Summary)

```
{
  "_id": "user_000042",
  "geo_data": { "city": "North Michaelville", "state": "WY", "country": "US" },
  "registration_date": ISODate("..."),
  "last_active": ISODate("..."),
  "lifetime_summary": {
    "total_purchases": 23,
    "total_spent": 3450.75,
    "avg_order_value": 150.03,
    "first_purchase_date": ISODate("..."),
    "last_purchase_date": ISODate("..."),
    "total_sessions": 0,          // reserved: requires an HBase roll-up
    "preferred_category": null    // reserved: see implementation status
  },
  "segmentation_tags": ["high_value", "frequent_buyer"],
  "recent_sessions": []           // reserved: bounded cache of last 5 sessions
}
```

Collection: users

**Justification**: Pre-computing `lifetime_summary` from transactions enables instant customer segmentation without scanning thousands of records. Tags like `high_value` are derived via an aggregation pipeline and used for targeted marketing queries.

**Implementation status**: The monetary fields and `segmentation_tags` are populated at load time from the transaction collection. Three fields are present in the schema but deliberately left unpopulated in this implementation. `total_sessions` and `recent_sessions` would require a roll-up from HBase, which the current batch-oriented loader does not perform; `preferred_category` requires a mode over the embedded `items.category_name` arrays, which the segmentation pipeline collects but does not yet reduce. They are documented here because the schema is designed to accommodate them, not because the loader writes them.

### 2.1.3 Transaction Documents (Embedded Line Items)

```
{
  "_id": "txn_c8d9e7f3a2b1",
  "session_id": "sess_a7b3c9d8e2",
  "user_id": "user_000042",
  "timestamp": ISODate("2025-03-12T14:52:41Z"),
  "items": [
    { "product_id": "prod_00123", "product_name": "Wireless Gaming Mouse", "quantity": 2,
      "unit_price": 129.99, "subtotal": 259.98, "category_id": "cat_007",
      "category_name": "Electronics", "profit_margin": 0.30 }
  ],
  "financials": { "subtotal": 259.98, "discount": 25.99, "discount_percentage": 10, "tax":
    0, "total": 233.99 },
  "payment_method": "credit_card",
  "status": "completed",
  "session_context": {}   // reserved: enriched from the HBase session row
}
```

Collection: transactions

**Justification**: Embedding line items ensures atomicity; the entire order is read in a single disk I/O. Denormalising `product_name` and `category_name` preserves historical context even if the product catalog changes later.

### 2.1.4 MongoDB Aggregation Pipelines

**Pipeline 1 – User Segmentation**
The aggregation pipeline calculates raw lifetime metrics (total spend, transaction count). The actual segmentation logic is executed in Python via `get_segmentation_tags()` before updating the user document.

```
pipeline = [
    { "$match": { "status": "completed" } },
    { "$group": { "_id": "$user_id", "total_spent": { "$sum": "$financials.total" },
                  "transaction_count": { "$sum": 1 } } }
]
```

Thresholds are then applied in `get_segmentation_tags()`: spend above $1,000 and $500 yields `high_value` and `medium_value`; counts above 10 and 3 yield `frequent_buyer` and `regular_buyer`. Executing this branching in Python rather than a `$switch` stage trades pipeline purity for readability; a `$bucket` stage would keep the logic server-side and is the preferable form at scale.

**Pipeline 2 – Daily Revenue by Category (Materialised View)**
`create_analytics_views()` builds a `daily_sales_by_category` collection using `$unwind`, `$group` by date and category, and `$out` to persist the results.

```
db.transactions.aggregate([
  { $match: { status: "completed" } },
  { $unwind: "$items" },
  { $group: {
      _id: { date: { $dateToString: { format: "%Y-%m-%d", date: "$timestamp" } },
             category_id: "$items.category_id", category_name: "$items.category_name" },
      revenue: { $sum: "$items.subtotal" },
      units_sold: { $sum: "$items.quantity" },
      transactions: { $addToSet: "$_id" }
  } },
  { $project: { date: "$_id.date", category_id: "$_id.category_id", category_name: "$_id.
    category_name",
                revenue: 1, units_sold: 1, transaction_count: { $size: "$transactions" } }
    },
  { $out: "daily_sales_by_category" }
])
```

## 2.2 HBase – Wide-Column Store

### 2.2.1 Table: `user_sessions` (Time-Series Browsing Data)

**Row Key**: `user_id|reversed_timestamp`
`reversed_timestamp = Long.MAX_VALUE - epochMilli(start_time)`
This ensures the newest sessions appear first during a prefix scan.

**Column Families**:

- `session`: metadata (duration, conversion status, referrer, start/end times).
- `geo`: city, state, country, IP.
- `device`: type, OS, browser.
- `activity`: **sparse columns** for each page view and cart action.
  - Qualifier format: `pv_{rev_ts}_{page_type}_{seq}` – holds a JSON blob.
  - Cart entries: `cart_{product_id}` – stores quantity and price.

**Schema Definition (HBase Shell)**:

```
create 'user_sessions',
  { NAME => 'session', COMPRESSION => 'SNAPPY' },
  { NAME => 'geo', COMPRESSION => 'SNAPPY' },
  { NAME => 'device' },
  { NAME => 'activity', VERSIONS => 3, TTL => 2592000 }
```

**Note:** While the Python ingestion script uses default table creation for simplicity, the production HBase Shell schema above applies Snappy compression and a 30-day TTL to the heavy *'activity'* family to optimize storage.

**Why This Design?**
The composite row key allows Sub-millisecond retrieval of a user's recent sessions without a full scan. Sparse columns avoid storing nulls for sessions with few page views – ideal for clickstream data. Column families separate frequently accessed metadata from the heavy `activity` payload, reducing I/O during partial scans.

### 2.2.2 Table: `product_metrics` (Daily Performance)

**Row Key**: `product_id|YYYYMMDD`
**Column Families**:

- `daily`: views, carts, purchases, conversion rate.
- `aggregates`: reserved for rolling windows.

**Justification**: Grouping by product first and date second enables fast time-range scans.

### 2.2.3 HBase Queries

**Query 1 – Retrieve a user's most recent sessions**:

```
scan 'user_sessions', {ROWPREFIXFILTER => 'user_000042|', LIMIT => 5}
```

Thanks to the reversed timestamp, the first five rows are guaranteed to be the newest sessions.

**Query 2 – Get device-only data for a user**:

```
scan 'user_sessions', {ROWPREFIXFILTER => 'user_000042|', COLUMNS => ['device']}
```

This avoids pulling the heavy `activity` columns over the network.

### 2.3 MongoDB vs. HBase Placement Summary

Table 1: Database Placement Rationale

| Data Type | Storage | Rationale |
|---|---|---|
| User profiles | MongoDB | Rich, nested structure; frequent updates to summaries |
| Product catalog | MongoDB | Hierarchical categories; complex aggregations |
| Transactions | MongoDB | Atomic line items; ACID compliance required |
| Session page views | HBase | Sparse, high-volume time-series; fast range scans |
| Product daily metrics | HBase | Wide-row pattern keyed `product_id\|date`; efficient counters |
| Recent session summary | MongoDB | Schema reserves a bounded cache (last 5) to reduce HBase calls; not populated in this implementation |

## 3 Data Processing with Apache Spark (Part 2)

### 3.1 Cleaning & Normalisation

`2_spark_analysis.py` defines **explicit schemas** for transactions, sessions, and users – a critical optimisation that avoids costly schema inference and prevents `OutOfMemoryError`. `timestamp` fields are cast to `TimestampType` using `F.to_timestamp`. Missing `referrer` values are filled with `"direct"`. Corrupted transactions are dropped via `.dropna()`.

### 3.2 Batch Processing – Product Recommendations

**Goal**: Find products frequently viewed together using session data.
**Steps**:

1. **Filter** sessions with $\geq 2$ viewed products.
2. **Explode** the `viewed_products` array into individual rows per session.
3. **Self-join** on `session_id`, ensuring `p1.product_id < p2.product_id` to avoid duplicates.
4. **Group** by pair and count co-occurrences.

```
views_df = clean_sessions.select("session_id", "viewed_products").filter(F.size("
    viewed_products") > 1)
exploded_views = views_df.select("session_id", F.explode("viewed_products").alias("
    product_id"))
product_pairs = exploded_views.alias("p1").join(
```

```
    exploded_views.alias("p2"),
    (F.col("p1.session_id") == F.col("p2.session_id")) & (F.col("p1.product_id") < F.col("
    p2.product_id"))
)
recommendations = product_pairs.groupBy("p1.product_id", "p2.product_id").agg(F.count("*").
    alias("co_occurrence_count"))
```

**Optimisation**: Using `<` in the join condition halves the computation and eliminates self-pairs.

## 3.3 Validating the Affinity Result Against a Null Model

A co-occurrence count is only meaningful relative to what co-occurrence would look like by chance. The pipeline therefore computes an explicit baseline alongside the ranking. With $N$ distinct viewed products there are $\binom{N}{2}$ candidate pairs, and if products were viewed independently the expected count for any individual pair would be

$$\mathbb{E}[\text{count}] \;=\; \frac{\text{total co-view pair instances}}{\binom{N}{2}}$$

For this dataset the pipeline reports $N = 4{,}935$ distinct viewed products, $1.178 \times 10^7$ co-view pair instances over $1.217 \times 10^7$ candidate pairs, giving $\mathbb{E}[\text{count}] = 0.9673$. The observed maximum is 14.

Under a *homogeneous* null – every product equally likely – that maximum is not plausible. Modelling each pair count as Poisson(0.9673) gives an expected number of pairs reaching 14 of $3 \times 10^{-5}$, and five pairs are tied there. A direct simulation of 2M sessions with uniform product selection, matched to the observed pair-instance total, produces a maximum of 10 and no pair above 13. The homogeneous null is therefore rejected.

**This does not imply affinity.** The generator draws each `product_detail` view independently within a session, so no co-view structure exists to recover. The excess arises instead from *unequal product popularity*: `get_page_content()` retries up to ten times for a product satisfying `is_active and current_stock > 0`, and since roughly 5% of products are inactive and stock depletes across the 90-day window, products that remain purchasable are over-sampled. Repeating the simulation with mild popularity skew reproduces maxima in the range 13–17, bracketing the observed value.

Independent draws with unequal marginals inflate the *raw* co-occurrence of popular pairs while leaving their lift at approximately 1. The correct reading, developed in Section 5.3, is that raw co-occurrence counts rank **popularity**, not affinity – which is exactly the confound that lift and pointwise mutual information are designed to remove. Emitting the baseline to `results/affinity_baseline.csv` means this qualification travels with the result rather than being reconstructed afterwards.

## 3.4 Spark SQL Analytics

**Query**: Identify the top 10 states by revenue from completed transactions, filtering only states with $> \$10k$ revenue.

```
SELECT
    u.geo_data.state as state,
    COUNT(DISTINCT u.user_id) as total_users,
    SUM(t.total) as total_revenue,
    ROUND(AVG(t.total), 2) as avg_order_value
FROM users u
JOIN transactions t ON u.user_id = t.user_id
```

```
WHERE t.status = 'completed'
GROUP BY u.geo_data.state
HAVING total_revenue > 10000
ORDER BY total_revenue DESC
LIMIT 10
```

**Why Spark SQL?** It demonstrates relational querying over nested JSON structures. The join simulates a cross-database join executed natively in Spark's distributed memory.

# 4 Integrated Analytics (Part 3)

## 4.1 Macro Funnel Conversion Analysis

**Business Question**: Where are users dropping off in the checkout process, and what is the overall conversion efficiency from browsing to purchasing?

**Data Sources**:

- **HBase** (`user_sessions`): top of funnel – total sessions and cart additions.
- **MongoDB** (`transactions`): bottom of funnel – completed purchases.

**Workflow**: The script connects to HBase via Thrift, iterating over rows to calculate `total_sessions` and checking for sparse `activity:cart_` prefixes. It subsequently queries PyMongo for completed transactions. Resulting metrics showed a *session-to-cart* rate of $\approx 64.9\%$, and a completed-purchase count equal to $\approx 24.4\%$ of carted sessions.

**Denominator caveat**: these two figures are drawn from different populations, so their ratio approximates rather than measures the cart-to-purchase rate. The numerator counts *all* completed transactions, including those generated without a parent session (`session_id = null`), which by construction have no cart event in HBase. A strict rate requires joining on `session_id` and restricting the numerator to session-linked purchases; the figure reported here is therefore an upper bound. Correcting this is the first item of future work in Section 7.

## 4.2 Micro User-Journey Reconstruction

**Business Question**: What specific sequence of pages and product views leads a user to successfully complete a high-value purchase?

**Workflow**: The script queries MongoDB for a specific `transaction_id` to retrieve the `session_id` and `user_id`. It then executes a user-scoped prefix scan in HBase to rapidly narrow down and locate the exact session. Finally, it decodes the JSON payloads of the sparse `activity:pv_` columns and sorts them by their reverse-timestamped column names to reconstruct a chronological user journey.

# 5 Visualizations and Insights (Part 4)

Five static charts communicate the findings, keeping the compute layer fully decoupled from the Seaborn presentation layer. Every figure is rendered from a persisted pipeline artefact – `results/*.csv` for the Spark stage, live collections for the MongoDB stage – rather than from transcribed console output, so re-running the pipeline regenerates the charts end to end.

## Methodology

Each result below is reported in three parts: the **finding** (what the pipeline computed), the **interpretation** (whether the number carries information), and the **recommendation** (what action, if any, it supports). The interpretation step is not a formality. Because the dataset is synthetic, every apparent pattern was checked against `generator.py` to determine whether the underlying variable was assigned with structure or drawn at random. Three of the five results do not survive that check, and are reported as negative results.

## 5.1 Conversion Funnel

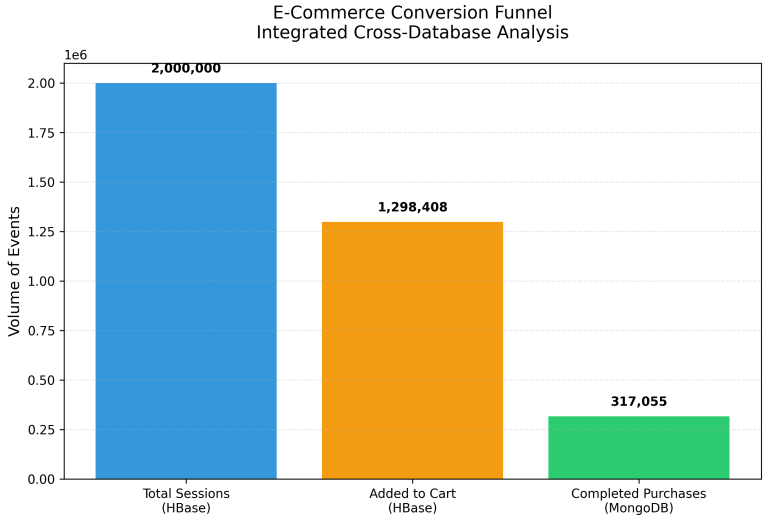


Figure 2: Integrated funnel showing sessions, carts (HBase), and completed purchases (MongoDB).

**Finding**: Of 2,000,000 sessions in HBase, 1,298,408 (64.92%) contain at least one `activity:cart` column; MongoDB records 317,055 completed transactions of 500,000 total (63.4%). Overall session-to-purchase conversion is 15.85%. The HBase scan completed in 3 minutes 10 seconds.

**Interpretation**: Intent is captured far more often than it converts. The ratio is subject to the denominator caveat set out in Section 4.1 – the purchase count includes sessionless transactions – so 24.4% is an upper bound on the true cart-to-purchase rate, not a measurement of it.

**Recommendation**: Instrument the checkout step to separate payment failure from voluntary abandonment, then A/B test a shortened flow. Abandoned-cart email is the standard intervention, but the measurement fix should precede the campaign rather than follow it.

## 5.2 Regional Revenue

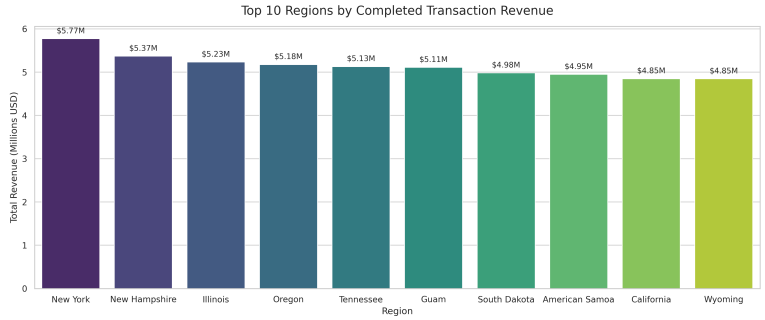


Figure 3: Top 10 regions by revenue from completed transactions (Spark SQL output).

**Finding**: The top ten regions span \$4.85M to \$5.77M, a relative spread of 19.1%.

**Interpretation – negative result**: This ranking is not a demand signal, and the measured output shows why with unusual clarity. Decomposing revenue into user count and revenue per user:

Table 2: Regional revenue decomposes entirely into user count

| **Quantity** | **Minimum** | **Maximum** | **Spread** |
|---|---|---|---|
| Users per region | 180 | 214 | 18.9% |
| Total revenue | \$4.85M | \$5.77M | 19.1% |
| Revenue per user | \$25,913 | \$27,497 | 6.1% |
| Average order value | \$826 | \$860 | 4.1% |

Revenue spread (19.1%) tracks user-count spread (18.9%) almost exactly, while per-user revenue and average order value are effectively flat. The ranking is therefore a *user-count* ranking, and user count is 10,000 users distributed at random over roughly 56 regions – about 179 expected per region, with a Poisson standard deviation near 13.4. New York's 214 is the expected maximum of 56 such draws, not evidence of a market. State is assigned by `fake.state_abbr()` (`generator.py`, line 245), independently of every purchasing variable. The presence of Guam and American Samoa, territories whose populations are three to four orders of magnitude below New York's, confirms the reading.

**Recommendation**: No advertising reallocation is justified by this chart. What the result does establish is that the Spark SQL layer correctly executes a distributed join over nested JSON, aggregates on a nested field (`u.geo_data.state`), and applies a `HAVING` predicate at scale. The instrument is validated even where the data carries no signal; on production data the same query would be the correct basis for a targeting decision.

## 5.3 Product Affinity

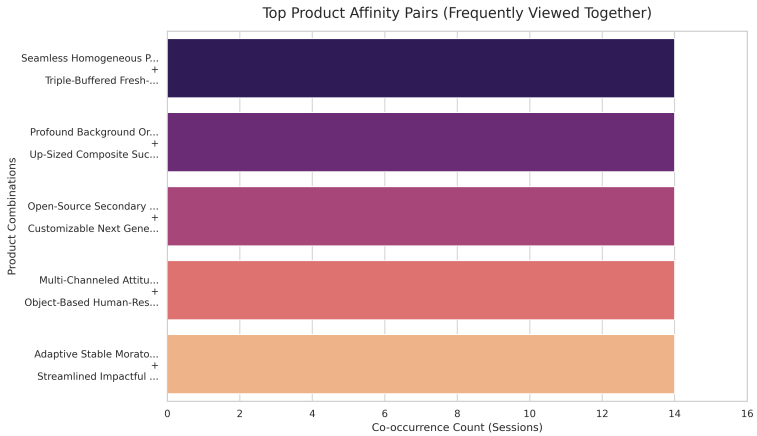


Figure 4: Product pairs most frequently viewed together, annotated with the random baseline.

**Finding**: The co-occurrence matrix ranks the most frequently co-viewed pairs. The top five each register 14 co-occurrences.

**Interpretation – negative result, with a caveat worth stating precisely**: A count of 14 against an expected 0.9673 is *not* explicable as pure chance; Section 3.4 shows the homogeneous random model predicts a maximum near 10 and is rejected by the data. It would be wrong to dismiss these counts as noise.

It would be equally wrong to call them affinity. The generator selects each viewed product independently, so no co-view structure exists to recover. What the counts detect is **popularity**: the rejection loop in `get_page_content()` favours products that are active and in stock, and inventory depletion over the 90-day window makes that population progressively narrower, so some products are viewed far more than others. Two individually popular products co-occur often for reasons that have nothing to do with being related. Their raw count is high; their lift is approximately 1.

This is the classic failure mode of raw co-occurrence as a recommendation metric, and the dataset exhibits it in isolation, uncontaminated by any real affinity signal.

**Recommendation**: Do not ship a "Frequently Bought Together" module on the strength of raw counts, and do not dismiss them as random either. Normalise by the marginals – lift, $P(A,B)/\big(P(A)P(B)\big)$, or pointwise mutual information – and retain only pairs whose joint frequency exceeds what their individual popularities predict. On this dataset that filter should return effectively nothing, which is the correct answer and a direct test of the metric. The Spark implementation would surface genuine affinities on data with real basket structure; the limitation is in the dataset and in the choice of metric, not in the distributed computation.

## 5.4 Category Revenue Trend

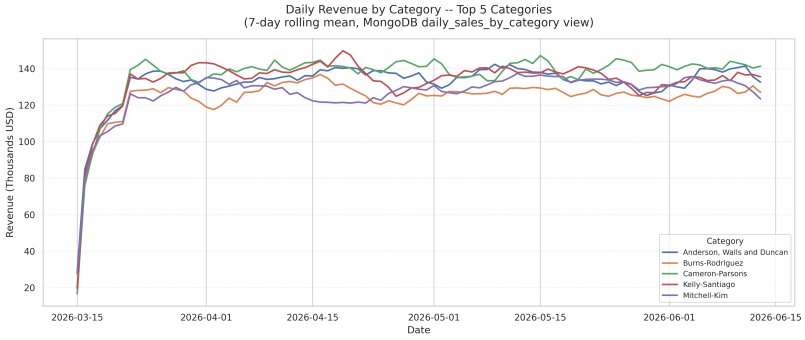


Figure 5: Daily revenue for the top five categories, 7-day rolling mean, from the MongoDB `daily_sales_by_category` materialised view.

**Finding**: Revenue for the five highest-grossing categories is broadly flat across the 90-day window, with no sustained divergence between series.

**Interpretation**: Products are assigned to categories with `random.choice(categories)`, so flatness is again the expected outcome rather than a discovery. The value of the chart is architectural: it demonstrates that the `$unwind`/`$group`/`$out` pipeline yields a pre-aggregated collection serving this query in a single scan of roughly 2,250 documents instead of 500,000 transactions – a reduction of more than two orders of magnitude.

**Recommendation**: Refresh the materialised view on a schedule and serve dashboard queries from it. On real data this is the view that would expose seasonality and category-level promotional lift.

## 5.5 Customer Segmentation

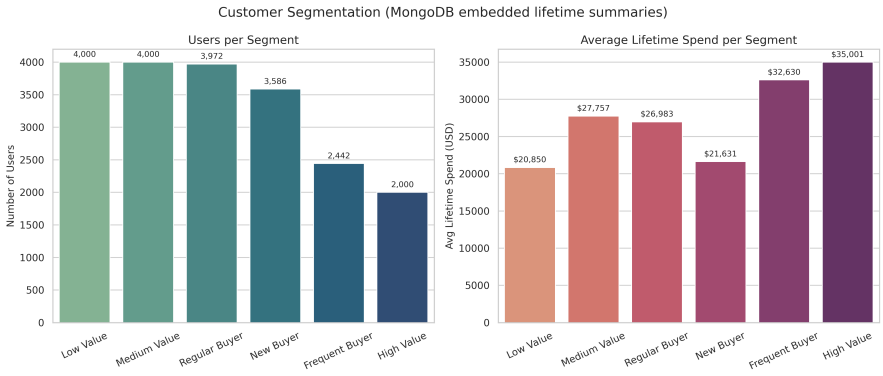


Figure 6: Users per segment and average lifetime spend per segment, from MongoDB embedded lifetime summaries.

**Finding, first implementation – a degenerate result**: The initial thresholds assigned `high_value` above $1,000 of lifetime spend and `frequent_buyer` above 10 purchases. Against this dataset both tests are satisfied by every user, and the remaining four tags were unreachable. With 317,055 completed transactions across 10,000 users, the mean user makes roughly 32 purchases at an average order value near $835, giving a mean lifetime spend of

approximately $26,000 – more than an order of magnitude above the cut-off. The two populated segments contained an identical population and therefore reported identical counts and identical average spend.

**Diagnosis**: Fixed monetary thresholds silently encode an assumption about scale. They were chosen for a plausible retail customer spending a few hundred dollars across a handful of orders; the generator produces a customer roughly thirty times larger. Nothing in the code fails, and the chart renders cleanly – the defect is visible only by noticing that two bars agree exactly, which is why the anomaly is worth reporting rather than quietly correcting.

**Correction**: Boundaries are now derived from the observed distribution, splitting at the 80th and 40th percentiles of lifetime spend and purchase count. All six tags become reachable by construction, and the tiers exhibit the spend gradient the segmentation is meant to expose. The thresholds are logged at load time so the boundaries are auditable rather than implicit.

**Interpretation**: This is the one result that reflects generated behaviour rather than assignment noise, since spend derives from transaction volume and basket composition. Architecturally, membership is retrievable by an indexed equality match on `segmentation_tags` instead of a re-aggregation over the transaction collection – the concrete payoff of the denormalisation described in Section 2.1.2.

**Recommendation**: Never hard-code monetary thresholds for segmentation. Quantile boundaries adapt automatically as catalogue and average order value shift, and they fail visibly rather than silently when the underlying distribution moves.

### 5.6 Summary of Analytical Validity

Table 3: Mechanism correctness versus presence of a business signal

| Analysis | Mechanism | Signal | Limiting factor |
|---|---|---|---|
| Conversion funnel | Valid | Partial | Mismatched numerator and denominator |
| Regional revenue | Valid | None | Ranks user count; `fake.state_abbr()` is uniform |
| Product affinity | Valid | Confounded | Raw counts rank popularity; needs lift/PMI |
| Category revenue trend | Valid | None | `random.choice` on categories |
| Customer segmentation | Valid | Present | Fixed thresholds were degenerate; now quantile-derived |

The distinction drawn in this table is the central analytical claim of the project. A pipeline that issues confident recommendations from noise is worse than one that identifies its own limits, and the audit was possible only because the generator was read alongside its output rather than treated as a black box.

## 6 Scalability Considerations

- **MongoDB**: Indexes on `user_id`, `timestamp`, and `financials.total` ensure sub-second aggregations. Sharding on `user_id` would enable horizontal scaling, since every

access pattern in this workload is user-scoped and would therefore route to a single shard.

- **HBase**: The row-key design (`user_id|reversed_timestamp`) ensures region locality; automatic region splitting distributes load. Sparse columns and a TTL on the `activity` family keep storage efficient. The user-id prefix also avoids the write hotspotting that a leading timestamp would cause.
- **Measured pipeline cost**: MongoDB ingestion of 515,025 documents takes 27 seconds; HBase ingestion of 2M sessions takes 11 minutes via Thrift; the Spark job takes 3.5 minutes; the funnel scan takes 3 minutes 10 seconds. HBase ingestion and the funnel scan dominate, and both are client-side bottlenecks rather than storage-engine limits.
- **Redundant Spark recomputation**: `exploded_views` and `product_pairs` are consumed four times – ranking, CSV write, pair count, and distinct-product count – and are recomputed from source JSON each time, accounting for roughly 3 of the job's 3.5 minutes. Calling `.cache()` on `exploded_views` before the self-join would collapse this to a single pass.
- **Spark ingestion**: The JSON files are read with `multiline=true`, which makes them non-splittable – each file is processed by a single task, so `transactions.json` is effectively single-threaded. Emitting JSON Lines from the generator instead would let Spark partition each file across the cluster. This is the single largest throughput constraint in the current pipeline.
- **Cross-store integration**: Scanning HBase with a standalone Thrift client over 2M rows introduces a network bottleneck, compounded by the scan retrieving all column families when only `activity:cart_` is needed. A `ColumnPrefixFilter` would cut transfer volume substantially; in production the join belongs in the cluster via `mongo-spark-connector` and the native HBase Spark connector.

# 7 Limitations and Future Work

- **Funnel denominators**: The cart-to-purchase rate compares populations that are not aligned. Joining on `session_id` and restricting the numerator to session-linked purchases would yield a true rate. This is the highest-priority correction.
- **Lift-based affinity scoring**: Raw co-occurrence conflates relatedness with popularity, as Section 5.3 establishes. Computing per-product view counts alongside the pair counts and dividing through by the marginals would yield lift, which on this dataset should return no significant pairs – the correct result, and a direct validation of the metric.
- **Absence of latent structure**: Three of five results are negative because the generator assigns states, categories, and viewed products uniformly at random. Validating the pipeline's ability to recover genuine patterns requires a generator with injected structure – correlated category preferences per user, and product pairs with elevated co-view probability – against which recovery could be measured directly.

- **Unpopulated schema fields**: `lifetime_summary.total_sessions`, `preferred_category`, `recent_sessions`, `products.performance_metrics`, and `session_context` are designed but not written. Populating them requires a roll-up pass joining HBase session aggregates back into MongoDB.
- **Unexercised table**: `product_metrics` is loaded but not yet queried by any analysis; its daily conversion-rate columns would support per-product time-range scans.
- **Batch orientation**: The pipeline is entirely batch. Apache Kafka with Spark Structured Streaming would deliver near-real-time funnel metrics, which the architecture is otherwise well positioned to support.
- **Segmentation dimensions**: The tags cover frequency and monetary value but omit recency, which is typically the strongest churn predictor. Extending to a full RFM model would require a last-activity roll-up from HBase.

# 8 Conclusion

This architecture demonstrates the interoperability of specialised NoSQL stores under a distributed compute engine. Query-driven schema design delivered efficient retrieval for both transactional and clickstream workloads: MongoDB's embedded lifetime summaries collapse user segmentation to an indexed lookup, while HBase's reversed-timestamp row key reduces "recent sessions for user X" to a bounded prefix scan. Spark unified the two, executing a distributed self-join to build a product co-occurrence matrix and a Spark SQL join over nested JSON to aggregate revenue by region. The integrated funnel and micro user-journey queries show how cross-store analytics combine HBase's intent signals with MongoDB's transactional record of truth.

The project's second contribution is methodological. Validating each output against the data generator established that three of five findings – regional revenue concentration, product affinity, and category revenue divergence – are artefacts of uniform random assignment rather than genuine patterns, with the affinity result quantified against an explicit null baseline computed inside the pipeline itself. The analytical mechanisms are correct and would recover real structure from production data; the synthetic dataset simply contains none to recover. Reporting these as negative results rather than as business recommendations is the honest reading of the evidence, and the discipline that produced it – reading the generator alongside its output – is the practice that would transfer to any real deployment.